

SMARTBRIDGE

Let's Bridge the Gap | SKILL WALLET - A SmartBridge Product

Date	29-09-2026
Team ID	SWTID-2026-7949
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/ServiceNow-Labs/Script-Controlled-ACL-Project-SWTID-2026-7804.git
Programming Language(s)	Server-Side JavaScript (ECMAScript 2021 / ES512 mode), XML (ServiceNow Update Sets)
Framework(s) Used	ServiceNow High Security Settings Framework, GlideSystem (gs) API, Access Control Engine
Key Features Implemented	• Custom table 'u_institution_details' schema with 7 data columns & test records. • Custom security roles: bb1 (Read), bb2 (Create), bb3 (Write), bb4 (Delete). • Server-side Script-Controlled Read ACL with Branch condition & gs.hasRole logic. • Granular Create, Write, and Delete ACLs assigned to specific custom roles. • Complete impersonation validation (Admin full access vs. EEE User branch isolation).
Pending / Incomplete Features	None. All milestones (User/Role Creation, Table Creation, and CRUD ACL implementation) are fully completed and verified.

Setup / Run Instructions:

1. Log into your ServiceNow Developer Instance with System Administrator credentials.
2. Elevate privileges by clicking Profile → 'Elevate role' and selecting 'security_admin'.
3. Set application scope and active update set to 'ACL Project (Global)'.
4. In User Administration, verify test user 'EEE User' and ensure roles bb1, bb2, bb3, bb4 are mapped.
5. Open System Security → Access Control (ACL) and confirm active Read, Create, Write, and Delete ACLs on table 'u_institution_details'.
6. Verify by impersonating 'EEE User': Navigate to 'u_institution_details.list' to confirm only EEE records appear with authorized CRUD actions.

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes

2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

- Server-side ACL evaluation prevents security bypass via client-side devtools, REST endpoints, and custom URL parameters.
- The implementation utilizes ServiceNow's recommended defense-in-depth model: combining table-level role requirement (bb1), data filter condition (Branch is EEE), and advanced JavaScript logic.
- Verified successfully with multi-persona impersonation tests in a dedicated Washington DC / Xanadu instance.